

Informatika 3

Prednáška 11: Koncepty

Michal Mrena



ŽILinskÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Vlastnosti typov



Typové parametre

- Šablóna je parametrizovaná typovým parametrom
- Uvádzame ho kľúčovým slovom `typename` alebo `class`
- Pri vytváraní inštancie šablóny zaň môžeme dosadiť ľubovoľný typ
 - Táto vlastnosť je zároveň výhodou aj nevýhodou
- Pri tvorbe kódu šablóny nemáme o skutočnom type žiadne informácie

```
template<typename T>
class DataFrame {
public:
    template<typename ResultType, typename Extractor>
    ResultType mean(Extractor extract) const;
};
```

```
template<class T>
class DataFrame {
    // ...
};
```

```
int main() {
    DataFrame<WeatherRecord> df;
    auto avg = df.mean<std::string>([](const auto &r){
        return r.temMin;
    });
}
```

Špecializácia šablón

- Istú možnosť prispôbovať implementáciu skutočným typom ponúkajú mechanizmy:
 - Čiastočnej špecializácie šablón
 - Úplnej špecializácie šablón
- Nevýhodou úplne špecializácie je že je až moc konkrétna – viedla by k veľkej duplicite kódu
 - Je však krokom k efektívnejšiemu riešeniu

```
template<typename T, typename U>
void my_swap(T &a, U &b) {
    T tmp = a;
    a = b;
    b = tmp;
}
```

```
template<>
inline void my_swap<int, int>(int &a, int &b) {
    a = b ^ a;
    b = a ^ b;
    a = b ^ a;
}
```

Špecializácia šablón

- Čiastočná špecializácia šablón je obmedzená na triedy
- Je limitovaný špeciálnymi prípadmi ako napríklad:
 - Počet typových parametrov
 - `const` a `volatile` kvalifikátormi
 - Typom ukazovateľ
 - Typom odkaz
- Môžeme imitovať čiastočnú špecializáciu nečlenských funkcií – avšak bez dedukcie typových parametrov

```
template<class T>
struct Printer {
    void print(std::ostream &ost, T val) const {
        ost << "value " << val << "\n";
    }
};

template<class T>
struct Printer<T*> {
    void print(std::ostream &ost, T *val) const {
        ost << "address " << val
            << " of value" << *val << "\n";
    }
};

int main() {
    int x = 10;
    Printer<int> pi;
    Printer<int*> ppi;
    pi.print(std::cout, x);
    ppi.print(std::cout, &x);
}
```

Špecializácia konštantných parametrov

- Šablóny môžeme špecializovať aj na základe konštantných (netypových parametrov)
- Špecializácia funguje rovnako ako pri typových parametroch

```
template<typename T, size_t Size>
class Array {
public:
    // ...

private:
    T m_data[Size];
};

template<typename T>
class Array<T, 0> {
    static_assert(false, "The size must be non-zero.");
};
```

Preskúmanie vlastností typov

- **Príklad:** Trieda `Serializer` v metóde `serialize` ukladá bajtovú reprezentáciu argumentu `value` do zoznamu bajtov `m_bytes`
- (od C++17) `std::byte` slúži na typovo silnejšiu reprezentáciu bajtov
- **Úloha:** Ak je `T` celočíselný typ, chceme ho serializovať iným spôsobom

```
class Serializer {  
public:  
    template<typename T>  
    void serialize(const T &value);  
  
    const std::vector<std::byte> &getBytes() const;  
  
private:  
    std::vector<std::byte> m_bytes;  
};
```

Preskúmanie vlastností typov

- Na zistenie, či je typ `T` celočíselný môžeme použiť mechanizmus úplnej špecializácie šablón
- Špecializáciou pre všetky celočíselné typy túto informáciu spoľahlivo získame
- Nevýhodou je väčšie množstvo kódu, množina celočíselných typov je však konečná a jasne definovaná

```
template<typename T>
struct IsIntegral {
    static const bool value = false;
};

template<>
struct IsIntegral<int> {
    static const bool value = true;
};

template<>
struct IsIntegral<long> {
    static const bool value = true;
};

// ...
```


Preskúmanie vlastností typov

- Statickú konštantu môžeme použiť na výber správnej implementácie
- Príkaz `if` je však vyhodnotený až za behu programu
- Informáciu o celočíselnosti typu však máme už počas prekladu – počas vytvárania inštancie šablóny

```
class Serializer {  
public:  
    template<typename T>  
    void serialize(const T &value) {  
        if (IsIntegral<T>::value) {  
            this->serializeIntegral(value);  
        } else {  
            this->serializeGeneric(value);  
        }  
    }  
  
private:  
    template<typename T>  
    void serializeIntegral(const T &value) const;  
  
    template<typename T>  
    void serializeGeneric(const T &value) const;  
};
```

Preťažovanie funkcií

- Pri volaní funkcie môže existovať niekoľko kandidátov s rovnakým názvom
- Títo tvoria množinu preťažení
- Kompilátor podľa pravidiel (napríklad minimalizácia konverzií) vyberie najvhodnejšiu funkciu alebo vyhlási chybu nejednoznačného vyhodnotenia preťaženia (angl. ambiguous overload)

```
template<typename T>
void print(const T &value) {
    std::cout << "Something else: " << value << "\n";
}

void print(double value) {
    std::cout << "A double: " << value << "\n";
}

void print(int value) {
    std::cout << "An int: " << value << "\n";
}

int main() {
    print(10);
    print(3.14);
}
```

Preťažovanie šablón funkcií

- Ak množina preťažení obsahuje šablóny funkcií, kompilátor vytvára inštancie šablón pomocou dedukovaných alebo explicitne špecifikovaných typov
- Ak pri vytváraní inštancie šablóny príde k chybe, nedochádza k chybe kompilácie, ale daná funkcia sa vyradí z množiny preťažení
- Tento mechanizmus sa označuje SFINE (angl. **S**ubstitution **F**ailure Is **N**ot An **E**rror)

```
template<typename T>
void print(const typename T::value_type &value) {
    std::cout << "Something with value: "
               << value.value << "\n";
}
template<typename T>
void print(const T &value) {
    std::cout << "Something else: " <<
               value << "\n";
}
void print(double value);
void print(int value);

int main() {
    print(2.71F); // Something else: 2.71
    print(10);    // An int: 10
    print(3.14);  // A double: 3.14
}
```

Preskúmanie vlastností typov

- Typ `EnableIf` je pomocný typ, ktorý je špecializovaný pre hodnoty `true` a `false`
- Iba špecializácia pre hodnotu `true` definuje členský alias `type` – identitu typového parametra
- Typ môžeme použiť v mechanizme SFINAE na definovanie čiastočnej špecializácie členskej funkcie

```
template<typename T, bool Value>
struct EnableIf;

template<typename T>
struct EnableIf<T, false> {
};

template<typename T>
struct EnableIf<T, true> {
    using type = T;
};
```

Čiastočná špecializácia členských funkcií

- Podľa hodnoty statickej konštanty `value` sa vyberie špecializácia šablóny `EnableIf`
- Ak sa vyberie špecializácia bez členského aliasu type, daná šablóna funkcie `serialize` sa odoberie z množiny preťažení

```
class Serializer {  
public:  
    template<typename T>  
    typename EnableIf<void, IsIntegral<T>::value>::type  
    serialize(const T &value) {  
        this->serializeIntegral(value);  
    }  
  
    template<typename T>  
    typename EnableIf<void, !IsIntegral<T>::value>::type  
    serialize(const T &value) {  
        this->serializeGeneric(value);  
    }  
};
```

Čiastočná špecializácia členských funkcií

```
class Serializer {  
public:  
    typename EnableIf<void, IsIntegral<int>::value>::type  
    serialize(const int &value) {  
        this->serializeIntegral(value);  
    }  
  
    typename EnableIf<void, !IsIntegral<int>::value>::type  
    serialize(const int &value) {  
        this->serializeGeneric(value);  
    }  
};
```

```
int main() {  
    Serializer s;  
    s.serialize(10);  
}
```

Čiastočná špecializácia členských funkcií

```
class Serializer {  
public:  
    typename EnableIf<void, true>::type  
    serialize(const int &value) {  
        this->serializeIntegral(value);  
    }  
  
    typename EnableIf<void, false>::type  
    serialize(const int &value) {  
        this->serializeGeneric(value);  
    }  
};
```

```
int main() {  
    Serializer s;  
    s.serialize(10);  
}
```

Čiastočná špecializácia členských funkcií

```
class Serializer {  
public:  
    void  
    serialize(const int &value) {  
        this->serializeIntegral(value);  
    }  
  
    typename EnableIf<void, false>::type  
    serialize(const int &value) {  
    this->serializeGeneric(value);  
    }  
};
```

```
int main() {  
    Serializer s;  
    s.serialize(10);  
}
```


Vlastnosti typov

- Štandardná knižnice obsahuje v hlavičke `<type_traits>` niekoľko tried na skúmanie vlastností typov
 - Okrem iných aj uvedené triedy `std::is_integral` a `std::enable_if`
- Samotná štandardná knižnica ich vo veľkom využíva
- Veľké uplatnenie majú v metaprogramovaní
- Ich syntax nie je používateľsky priateľská
- Ešte menej prívetivé sú chybové výpisy

Limity

- Špecializáciu šablón využíva štandardná knižnica aj na iných miestach
- Jedným z príkladov je knižnica `<limits>`, ktorá sprístupňuje – okrem iného – minimálne a maximálne hodnoty číselných typov

```
int main() {  
    std::cout << "Max value of int is "  
        << std::numeric_limits<int>::max() << ".\n";  
    std::cout << "Max value of double is "  
        << std::numeric_limits<double>::max() << ".\n";  
    std::cout << "Min value of double is "  
        << -std::numeric_limits<double>::min() << ".\n";  
}
```

Vlastnosti iterátorov

- Ďalším príkladom špecializácie je trieda `std::iterator_traits`
- Jej úlohou je sprístupniť vlastnosti iterátorov, ktoré sú typicky členmi iterátorov
- Výnimkou je typ ukazovateľ, ktorý je plnohodnotným iterátorom s náhodným prístupom – nemá však členské aliasy typov

```
template<typename InputIt>
typename InputIt::value_type
sum(
    InputIt first,
    InputIt last
) {
    using T = typename InputIt::value_type;
    T result{};
    for (; first != last; ++first) {
        result += *first;
    }
    return result;
}

int main() {
    std::forward_list<int> xs{1,2,3,4};
    std::cout << sum(xs.begin(), xs.end()) << "\n";
}
```

Vlastnosti iterátorov

- Ďalším príkladom špecializácie je trieda `std::iterator_traits`
- Jej úlohou je sprístupniť vlastnosti iterátorov, ktoré sú typicky členmi iterátorov
- Výnimkou je typ ukazovateľ, ktorý je plnohodnotným iterátorom s náhodným prístupom – nemá však členské aliasy typov

```
template<typename InputIt>
typename InputIt::value_type
sum(
    InputIt first,
    InputIt last
) {
    using T = typename InputIt::value_type;
    T result{};
    for (; first != last; ++first) {
        result += *first;
    }
    return result;
}

int main() {
    int xs[]{1,2,3,4};
    std::cout << sum(xs, xs+4) << "\n";
}
```

Vlastnosti iterátorov

- Ďalším príkladom špecializácie je trieda `std::iterator_traits`
- Jej úlohou je sprístupniť vlastnosti iterátorov, ktoré sú typicky členmi iterátorov
- Výnimkou je typ ukazovateľ, ktorý je plnohodnotným iterátorom s náhodným prístupom – nemá však členské aliasy typov

```
template<typename InputIt>
typename std::iterator_traits<InputIt>::value_type
sum(
    InputIt first,
    InputIt last
) {
    using T =
        typename std::iterator_traits<InputIt>::value_type;
    T result{};
    for (; first != last; ++first) {
        result += *first;
    }
    return result;
}

int main() {
    int xs[]{1,2,3,4};
    std::cout << sum(xs, xs+4) << "\n";
}
```

Konštantné výrazy



Konštantné výrazy

- Použitie príkazu `if` je jednoduché, čitateľné a zrozumiteľné
- Jedinou nevýhodou je jeho vyhodnocovanie až za behu programu

```
class Serializer {  
public:  
    template<typename T>  
    void serialize(const T &value) {  
        if (IsIntegral<T>::value) {  
            this->serializeIntegral(value);  
        } else {  
            this->serializeGeneric(value);  
        }  
    }  
}  
  
private:  
    template<typename T>  
    void serializeIntegral(const T &value) const;  
  
    template<typename T>  
    void serializeGeneric(const T &value) const;  
};
```

Konštantné výrazy

- Použitie príkazu `if` je jednoduché, čitateľné a zrozumiteľné
- Jedinou nevýhodou je jeho vyhodnocovanie až za behu programu
- (od C++17) Je možné použiť príkaz `if constexpr`, ktorý sa vyhodnocuje počas prekladu

```
class Serializer {  
public:  
    template<typename T>  
    void serialize(const T &value) {  
        if constexpr (IsIntegral<T>::value) {  
            this->serializeIntegral(value);  
        } else {  
            this->serializeGeneric(value);  
        }  
    }  
  
private:  
    template<typename T>  
    void serializeIntegral(const T &value) const;  
  
    template<typename T>  
    void serializeGeneric(const T &value) const;  
};
```


Konštantné výrazy

- Podmienka príkazu `if constexpr` musí byť konštantný výraz
- Typickým príkladom takéhoto výrazu sú celočíselné konštanty (vrátane typu `bool`)
- Špecifikátorom `constexpr` môžeme označiť premennú
 - Automaticky implikuje `const`
- Špecifikátorom `constexpr` môžeme označiť funkciu
 - Automaticky implikuje `inline` (aj pri statickej členskej premennej)
 - Volanie takejto funkcie môže byť vyhodnotené počas už prekladu (ak sú argumenty konštantné výrazy)

Konštantné výrazy

- `constexpr` funkcie je možné používať počas behu programu ako obvykle
- Ak funkcia/premenná môže byť `constexpr`, je výhodné ju tak označiť
- `constexpr` funkcie môžu byť aj zložitejšie, môžu dokonca alokovať pamäť dynamicky

```
template<typename T, int Size>
class Array {
public:
    int size() const {
        return Size;
    }
};

constexpr int factorial(int n) {
    int result = 1;
    for (int i = 2; i <= n; ++i) {
        result *= i;
    }
    return result;
}

int main() {
    Array<int, factorial(5)> xs;
    std::cout << xs.size() << "\n"; // 120
    std::cout << factorial(6) << "\n"; // 720
}
```

Podmienená kompilácia

- Príkazom `if constexpr` môžeme dosiahnuť preloženie/volbu iba vybranej časti kódu, nemôžeme ho však použiť mimo tela funkcie
- Pod pojmom podmienený preklad však rozumieme aj podmienené vynechanie časti kódu vo fáze predspracovania
- Tento mechanizmus je jednoduchší a nízko-úrovňovejší – nijako nepracuje s typovým systémom
- Často sa s ním stretávame v multiplatformových knižniciach

```
#ifdef __WIN32
#include <windows.h>
#endif

int main(int argc, char *argv[]) {
#ifdef __WIN32
    SetConsoleOutputCP(CP_UTF8);
#endif
    // ...
}
```

Koncepty



Koncepty

- Preskúmaním vlastnosti typov dokážeme ovplyvniť množinu preťažení
- Dokážeme tiež podmieniť vlastnosti typového parametra, syntax je však pomerne komplikovaná
- (od C++20) Môžeme na oboje použiť nový prvok jazyka – koncepty
- Koncept definuje množinu vlastností typového parametra

```
template<{template-parameter-list}>  
concept {concept-name} = constraint-expression;
```

`constraint-expression` je konštantný výraz, ktorý
sa vyhodnotí na hodnotu typu `bool`.*

```
template<typename T>  
concept Integral = IsIntegral<T>::value;
```

V definícii mnohých konceptov sú použité
existujúce traity.

Táto definícia nie je úplne presná, ale pre naše potreby je dostatočne presná. *

Použitie konceptov

- Názov konceptu uvádzame namiesto kľúčového slova `typename`
- Pri jeho použití uvádzame o jeden typový parameter menej – je dedukovaný automaticky
- Konceptom môžeme ovplyvniť množinu preťažení
- Kompilátor vyberie najšpecifickejšiu implementáciu

```
class Serializer {  
public:  
    template<Integral T>  
    void serialize(const T &value) {  
        this->serializeIntegral(value);  
    }  
  
    template<typename T>  
    void serialize(const T &value) {  
        this->serializeGeneric(value);  
    }  
};
```

```
int main() {  
    Serializer s;  
    s.serialize(10); // Serializing integral.  
    s.serialize(3.14); // Serializing generic.  
}
```

Použitie konceptov

- Koncepty môžeme použiť aj bez preťažovania
- Môžeme nimi upresniť nutné vlastnosti typového parametra
- Okrem nahradenia kľúčového slova `typename` ich môžeme uviesť samostatne kľúčovým slovom `requires`
- V tomto prípade explicitne uvádzame typový parameter

```
template<typename Base, typename Exponent>
requires Integral<Exponent>
constexpr Base int_pow(Base base, Exponent exponent) {
    Base result{1};
    for (;;) {
        if (exponent & 1U) {
            result *= base;
        }

        exponent >>= 1U;

        if (0 == exponent) {
            break;
        }

        base *= base;
    }
    return result;
}
```

Podmienená kompilácia

- Koncepty môžeme použiť ako podmienku podmieneného prekladu
- V tomto prípade explicitne uvádzame typový parameter

```
class Serializer {  
public:  
    template<typename T>  
    void serialize(const T &value) {  
        if constexpr (Integral<T>) {  
            this->serializeIntegral(value);  
        } else {  
            this->serializeGeneric(value);  
        }  
    }  
};  
  
private:  
    template<typename T>  
    void serializeIntegral(const T &value) const;  
  
    template<typename T>  
    void serializeGeneric(const T &value) const;  
};
```


Podmienená kompilácia

```
template<typename Degree, typename Domain>
template<out_var_values Vars, std::output_iterator<Vars> OutputIt>
void diagram_manager<Degree, Domain>::satisfy_all_g(
    const int32 value,
    const diagram_t &diagram,
    OutputIt out
) const {
    if constexpr (domains::is_fixed<Domain>::value) {
        assert(value < Domain::value);
    }

    Vars vars;
    if constexpr (tools::is_std_vector<Vars>) {
        vars.resize(as_usize(this->get_var_count()));
    }

    node_t *const root = diagram.unsafe_get_root();
    this->satisfy_all_impl(value, vars, out, root, 0);
}
```

Tvorba konceptov

- Pri definícii nových konceptov môžeme použiť konjunkciu a disjunkciu existujúcich konceptov a konštantných výrazov
- Koncepty nesmú byť rekurzívne
- V definícii ich typového parametra nie je možné použiť iné koncepty

```
template<typename _Ty>
concept integral = is_integral<_Ty>::value;

template<typename _Ty>
concept signed_integral =
    integral<_Ty> &&
    static_cast<_Ty>(-1) < static_cast<_Ty>(0);

template<typename _Ty>
concept unsigned_integral =
    integral<_Ty> &&
    !signed_integral<_Ty>;
```

Zjednodušená definícia konceptov
z hlavičky `<concepts>`.

Tvorba konceptov

- Pri tvorbe zložitejších konceptov môžeme použiť výraz `requires`*
- Môže obsahovať (pseudo)parametre
- V tele môžeme testovať prítomnosť členských funkcií a typov
- Ak sú všetky riadky korektné (daný člen existuje a výraz je správny) celý výraz `requires` nadobúda hodnotu `true`, inak `false`

```
template<typename C>
concept Container = requires(C c) {
    typename C::value_type;
    c.clear();
    c.size();
    c.begin();
    c.end();
};
```

Iný význam kľúčového slova ako pri aplikácii konceptu. *

Tvorba konceptov

- Vo výraze môžeme kontrolovať aj návratové typy
- Môžeme na ne aplikovať koncepty
- Štandardná knižnica obsahuje koncepty pokrývajúce bežné situácie v hlavičke `<concepts>`, v hlavičke `<iterator>` a v hlavičke `<ranges>`

```
template<typename C>
concept Container = requires(C c) {
    typename C::value_type;
    { c.clear() } -> std::same_as<void>;
    { c.size() } -> std::convertible_to<std::size_t>;
    { c.begin() } -> std::input_iterator;
    { c.end() } -> std::input_iterator;
};
```

Tvorba konceptov

```
template<typename P, typename T>
concept Predicate = requires(P test, T t) {
    { test(t) } -> std::convertible_to<bool>;
};
```

```
template<typename T>
class DataFrame {
public:
    template<Predicate<T> P>
    std::vector<const T*> filter(P satisfies) const;
};
```

```
int main() {
    DataFrame<int> df;
    df.filter([](const int &x){
        return std::string("yes");
    });
}
```

Tvorba konceptov

```
template<typename T>
concept Arithmetic = requires(T a, T b) {
    { a + b } -> std::convertible_to<T>;
    { a - b } -> std::convertible_to<T>;
    { a * b } -> std::convertible_to<T>;
    { a / b } -> std::convertible_to<T>;
};
```

```
template<typename E, typename T, typename V>
concept ExtractorOf = requires(E extract, T t) {
    { extract(t) } -> std::convertible_to<V>;
};
```

```
template<typename T>
class DataFrame {
public:
    template<typename ValueType, typename Extractor>
    requires Arithmetic<ValueType> && ExtractorOf<Extractor, T, ValueType>
    ValueType mean(Extractor extract) const;
};
```

Konceptom definované rozhrania

- Koncepty nám umožňujú definovať *rozhranie*, ktoré je kontrolované počas prekladu
- Ide o iný prístup k implementácii polymorfizmu
- Prípady použitia sa čiastočne prekrývajú s rozhraniami (abstraktnými triedami)
- Mechanizmus podobný traitom jazyka Rust a typovým triedam jazyka Haskell

```
template<typename T>
concept Printable = requires(T t, std::ostream &ost) {
    { t.to_string() } -> std::convertible_to<std::string>;
    { ost << t } -> std::same_as<std::ostream&>;
};
```

```
pub trait Printable: fmt::Display + ToString {}
```

Trait v jazyku Rust.

```
class Printable a where
    toString :: a -> String
    toStream :: a -> IO ()
```

Typová trieda v jazyku Haskell.

Rozsahy



Rozsahy

- Viaceré programovacie jazyky umožňujú spracovanie údajov **deklaratívnym spôsobom**
- Operácie definujú nad zovšeobecnením kontajnera – nad **rozsahom** (angl. **range**)
- Tento spôsob spracovania je známy aj ako **filter—map—reduce**
- Ide o reťazenie uvedených operácií – funkcií vyššieho rádu

Jazyk C#

```
class Program {  
    static void Main() {  
        List<int> data = new List<int> { 1, 2, 3, 4, 5 };  
        double mean = data.Average();  
        double variance = data  
            .Select(x => Math.Pow(x - mean, 2))  
            .Average();  
        Console.WriteLine($"Mean: {mean}");  
        Console.WriteLine($"Variance: {variance}");  
    }  
}
```

V jazyku C# môžeme na deklaratívne spracovanie údajov použiť knižnicu [LINQ](#).

Jazyk Java

```
public class Main {  
    public static void main(String[] args) {  
        List<Integer> data = Arrays.asList(1, 2, 3, 4, 5);  
        double mean = data.stream()  
            .reduce(0, Integer::sum) / (double)data.size();  
        double variance = data.stream()  
            .map(x -> Math.pow(x - mean, 2))  
            .reduce(0.0, Double::sum)  
            / data.size();  
        System.out.println("Mean = " + mean);  
        System.out.println("Variance = " + variance);  
    }  
}
```

V jazyku Java môžeme na deklaratívne spracovanie údajov použiť rozhranie Stream.

Jazyk JavaScript

```
const data = [1, 2, 3, 4, 5];
const mean = data.reduce((acc, x) => acc + x, 0)
  / data.length;
const variance = data
  .map(x => Math.pow(x - mean, 2))
  .reduce((acc, x) => acc + x, 0)
  / data.length;
console.log("Mean:", mean);
console.log("Variance:", variance);
```

V jazyku JavaScript je deklaratívne spracovanie súčastou rozhrania typu Array.

Jazyk Haskell

```
main :: IO ()
main = do
  let xs = [1, 2, 3, 4, 5]
  let n = fromIntegral (length xs)
  let mean = foldl (+) 0 xs / n
  let variance = foldl (+) 0 (map (\x -> (x - mean) ^ 2) xs) / n
  putStrLn $ "Mean: " ++ show mean
  putStrLn $ "Variance: " ++ show variance
```

V jazyku Haskell je deklaratívne spracovanie inherentnou súčasťou jazyka

Rozsahy v jazyku C++

- (od C++20) V jazyku C++ túto funkcionálnosť poskytuje knižnica `<ranges>`
- Staršia knižnica `<algorithm>` definuje algoritmy voči iterátorom
- Knížnica `<ranges>` definuje algoritmy voči konceptu `std::range`
 - Podobne ako pri iterátoroch existujú rozsahy s rôznymi vlastnosťami (vstupný, výstupný, obojsmerný, s náhodným prístupom, ...)
- Typ spĺňajúci tento koncept musí poskytovať iterátory (`begin`, `end`)
- Reťazenie operácií na rozsahmi je zabezpečené preťažením operátora `|`
- Ich použitie podrobnejšie demonštruje tento článok

Rozsahy v jazyku C++

```
int main() {
    std::vector<int> data {1, 2, 3, 4, 5};

    const double n = static_cast<double>(data.size());
    const double mean = std::reduce(data.begin(), data.end(), 0.0) / n;

    const auto devsSq = data
        | std::ranges::views::transform([mean](int x) { return (x - mean); })
        | std::ranges::views::transform([mean](const auto x) { return x*x; });

    const double variance = std::reduce(devsSq.begin(), devsSq.end(), 0.0) / n;

    std::cout << "Mean = " << mean << "\n";
    std::cout << "Variance = " << variance << "\n";
}
```

Knižnica `<ranges>` nanešťastie neobsahuje algoritmus `reduce`. *

Zhmotnenie rozsahu

- (od C++23) Môžeme rozsah transformovať naspäť na kontajner:

```
template<typename T>
class DataFrame {
public:
    template<Predicate<T> P>
    std::vector<const T*> filter(P satisfies) const {
        return m_data
            | std::ranges::views::filter(satisfies)
            | std::ranges::views::transform([](const T &x) { return &x; })
            | std::ranges::to<std::vector<const T*>>();
    }

private:
    std::vector<T> m_data;
};
```


Lenivé vyhodnocovanie

- Reťazenie rozsahov používa lenivé vyhodnocovanie – filtrovanie, mapovanie a iné operácie sa vykonávajú, až keď potrebujeme prvky rozsahu sprístupniť

```
int main() {  
    std::vector<int> data = {1, 2, 3, 4, 5};  
  
    const auto xs = data  
        | std::ranges::views::filter([](int x){ return x % 2 == 1; })  
        | std::ranges::views::transform([](int x){ return x * x; });  
  
    // Operácie filtrovania a transformácie  
    // sa vykonávajú až pri sprístupnení prvkov.  
    for (const int x : xs) {  
        std::cout << "value: " << x << "\n";  
    }  
}
```

Zhrnutie

- Koncepty značne uľahčujú a sprehľadňujú prácu s typovými parametrami
- Existujúci kód napísaný v staršom štandardne obsahuje použitie `<type_traits>`
- `<type_traits>` majú zmysel aj v štandardne C++20
- Knižnica `<ranges>` umožňuje deklaratívne spracovanie údajov

Ďakujem za pozornosť

Priestor na otázky

